Assignment 1: Design Patterns – Console App Practice

Dear Students 👋

After our first lecture on **Design Patterns**, your task is to implement the following three questions using **C# Console Applications**.

Each question focuses on one design pattern:

1. **Dependency Injection (DI)**
2. **Singleton**
3. **Builder**

Please read the requirements carefully and submit clean, well-formatted code with clear console output.

Question 1: Dependency Injection 💉

Create a console app for a Tax Calculation System

Problem Description

Build a system where `InvoiceService` calculates the final invoice total using different tax services such as:

- `EgyptTaxService`
- `SaudiTaxService`

Requirements

- Create an interface like `ITaxService` with a method to calculate tax
- Create two classes that implement it:
  - `EgyptTaxService` (e.g., 14% tax rate)
  - `SaudiTaxService` (e.g., 15% tax rate)
- Create an `InvoiceService` class that depends on `ITaxService`
- Inject the tax service into `InvoiceService` through the constructor

- In `Main()`, test the system with both tax services
- Print the subtotal, tax amount, and final total in the console

Goal

🎯 Learn how to make the service depend on an **interface** instead of creating tax classes directly. This reduces coupling and makes your code easier to test and maintain.

Expected Output Example

```
--- Egypt Tax System ---
Subtotal: 1000 EGP
Tax (14%): 140 EGP
Total: 1140 EGP

--- Saudi Tax System ---
Subtotal: 1000 SAR
Tax (15%): 150 SAR
Total: 1150 SAR
```

Question 2: Singleton 🔒

Create a console app for a Company Printer Queue

Problem Description

Build a class called `PrinterManager` that has only **one instance** in the whole application. This ensures that all parts of the program share the same printer resource.

Requirements

- Prevent object creation from outside the class (make constructor private)
- Create a static method or property to return the same instance every time
- Add a method like `PrintDocument(string documentName)` that prints to console
- Add a counter to track how many documents have been printed
- In `Main()`, create multiple variables and prove they all use the same printer manager object
- Print at least 3 different document names in the console

Goal

🎯 Understand how **Singleton** is useful when the whole system should share one common resource (like a printer, database connection, or logger).

Expected Output Example

```
Printer initialized.
Printing: Report.pdf (Total printed: 1)
Printing: Invoice.docx (Total printed: 2)
Printing: Contract.pdf (Total printed: 3)

Testing if same instance:
printer1 == printer2: True
printer1 == printer3: True
```

Question 3: Builder 📐

Create a console app for building a Computer object step by step

Problem Description

Build a `Computer` class with multiple optional properties. Instead of using a constructor with many parameters (which is confusing), use the **Builder Pattern** to create computer objects in a clean and readable way.

Requirements

- Create a `Computer` class with properties such as:
  - Processor
  - RAM (in GB)
  - Storage (in GB)
  - HasGraphicsCard (bool)
  - OperatingSystem
- Create a `ComputerBuilder` class with methods like:
  - `SetProcessor(string processor)`
  - `SetRAM(int ram)`
  - `SetStorage(int storage)`
  - `SetGraphicsCard(bool hasGPU)`
  - `SetOperatingSystem(string os)`
- Add a `Build()` method that returns the final `Computer` object
- In `Main()`, create at least two different computers using the builder
- Print the computer specifications to the console

Goal

🎯 Practice creating complex objects in a **clean and readable way** without constructor confusion.

Expected Output Example

--- Gaming Computer ---
Processor: Intel i9
RAM: 32 GB
Storage: 1000 GB
Graphics Card: Yes
OS: Windows 11

--- Office Computer ---
Processor: Intel i5
RAM: 8 GB
Storage: 256 GB
Graphics Card: No
OS: Windows 10

Submission Guidelines

Bonus Points ⭐

Extra credit for students who:

- Add input validation
- Create more than the minimum required classes
- Write exceptionally clean and well-organized code
- Add extra features that demonstrate deep understanding

Link for Drive

https://drive.google.com/drive/folders/10J2c8ytMfj1d-zXu6whAA7_go14HT8m8?usp=sharing

---

Good luck! 🚀

If you have any questions, feel free to ask.

Happy Coding! 👩‍💻 👨‍💻